

International Institute of Information Technology, Bhubaneswar

Project Abstract

Project title

Text Editor in C++ using FLTK and OOP

Name: Roushan Sharma

Branch: CSE-B

Roll No.: B125103

Subject: Object-Oriented Programming (OOPs)

Department: Computer Science

Abstract

This project is a desktop text editor written in C++ with the FLTK (Fast Light Toolkit) graphical user interface library, designed using object-oriented programming. Users can create, open, edit and save plain text files through a menu bar and keyboard shortcuts. The editor supports cut, copy, paste, delete and undo, along with find, find next, replace and replace all.

The editor records whether the file has unsaved changes, shows "(modified)" in the title bar, and asks the user to save before closing the file or exiting. It also provides syntax highlighting for C and C++ source code, displaying keywords, data types, strings, comments and preprocessor directives in different colours and font styles.

The program is divided into classes for the main window, the document, file input and output, search and replace, and syntax highlighting. This structure applies encapsulation, inheritance, abstraction and polymorphism, so that each part of the program can be modified or extended without changing the others.

Introduction and problem statement

A text editor is one of the most frequently used programs on a computer. Widely used editors contain a very large amount of code, so their internal design is difficult to study. This project builds a small editor that still covers the essential features, so that its complete design can be understood and explained.

An editor of this kind can be written with global variables for the text buffer, the file name and the modified flag, together with separate C-style callback functions. That approach works, but every function depends on shared global state, which makes the code difficult to extend, test and reuse.

This project therefore has two aims: to build a working, menu-driven text editor, and to design it with object-oriented principles so that data is encapsulated in classes, each class has a single responsibility, and new features can be added without rewriting existing code.

Objectives

1. Provide a menubar with File, Edit and Search menus and keyboard shortcuts for every function.
2. Create a new file, open an existing file, insert a file, save, and save as.

3. Support cut, copy, paste, delete and undo on selected text.
4. Provide find, find next, replace next and replace all through a dedicated dialog.
5. Track whether the file has changed, show it in the title bar, and prompt the user to save before discarding changes.
6. Highlight C/C++ syntax using a separate style buffer.
7. Structure the code into classes that clearly demonstrate the four pillars of OOP.

Scope

In scope: a single-window desktop editor for plain text files, with one document open at a time, the editing and search features listed above, unsaved-change protection, and C/C++ syntax highlighting.

Out of scope: multiple tabs or documents, rich text formatting (bold, fonts, images), printing, spell checking, plugins, and network or cloud storage. These are listed as future enhancements.

Tools and technologies

Item	Choice	Purpose
Language	C++ (C++17)	Core implementation using classes
GUI library	FLTK 1.3 or 1.4	Windows, menus, text editor widget, dialogs
Key FLTK classes	`Fl_Double_Window`, `Fl_Menu_Bar`, `Fl_Text_Editor`, `Fl_Text_Buffer`	Building blocks of the editor
Compiler	g++ / MinGW / MSVC	Building the executable
Build	`fltk-config` or CMake	Compiling and linking FLTK
IDE	VS Code or Code::Blocks	Writing and debugging code
Version control	Git	Tracking progress phase by phase
Platform	Windows or Linux	FLTK is cross-platform

Object-oriented design

Instead of global variables and free functions, the editor is built from seven classes, each with one clear responsibility.

Class	Inherits from	Responsibility
`EditorApp`	None	Starts the program, owns the document and the window, runs the event loop
`EditorWindow`	`Fl_Double_Window`	Holds the menu bar and the `Fl_Text_Editor` widget, handles menu actions
`Document`	None	Stores the text buffer, file name and modified flag; builds the window title
`FileManager`	None	Loads and saves files, reports read and write errors
`SearchReplaceDialog`	`Fl_Window`	Find and replace inputs and buttons; find next, replace next, replace all
`SyntaxHighlighter` (abstract)	None	Interface with a pure virtual `parse()` method for styling text
`CppHighlighter`	`SyntaxHighlighter`	Styles C/C++ keywords, types, strings, comments and directives

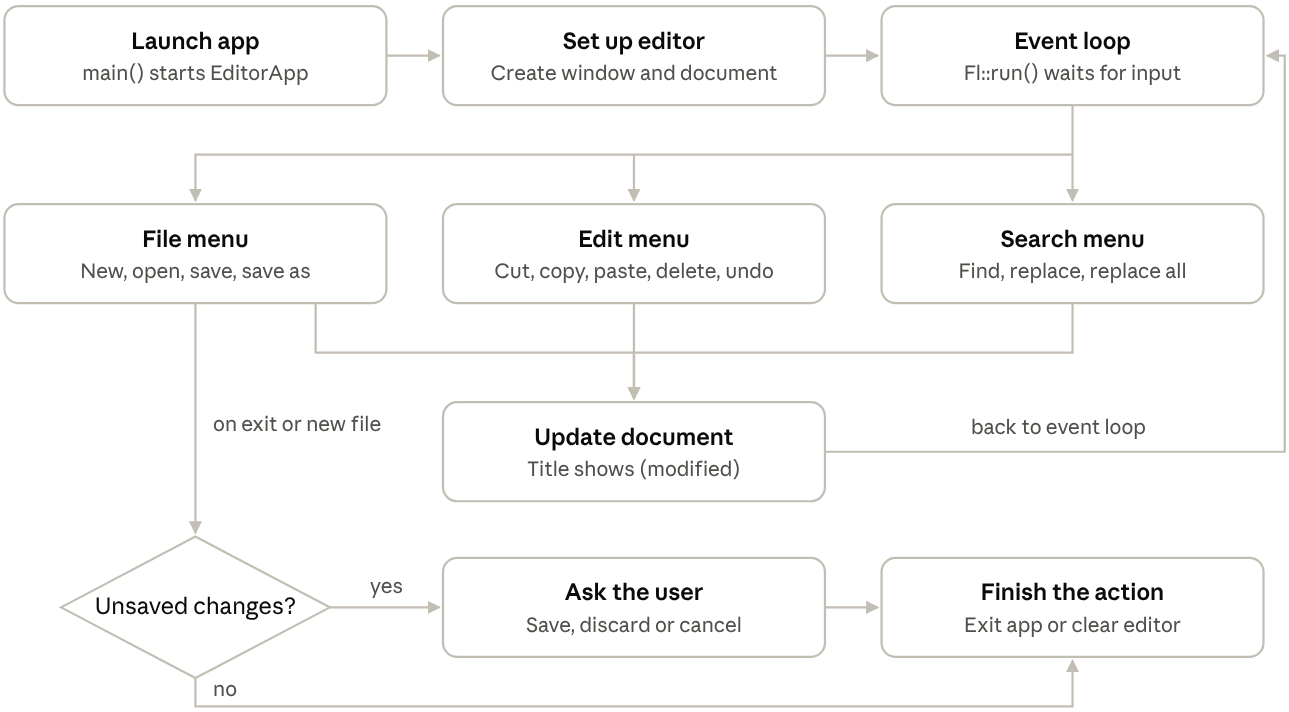
How the four pillars appear:

- **Encapsulation:** `Document` keeps the buffer, filename and modified flag private, exposing only methods such as `isModified()`, `setFilename()` and `getTitle()`.
- **Inheritance:** `EditorWindow` and `SearchReplaceDialog` extend FLTK window classes; `CppHighlighter` extends `SyntaxHighlighter`.
- **Abstraction:** `SyntaxHighlighter` defines what a highlighter does without saying how, so the editor depends only on the interface.
- **Polymorphism:** the window calls `highlighter->parse()` through a base-class pointer, so a new language highlighter can be added without changing the window.

Design challenge: FLTK callbacks must be static or free functions. Each class uses a static member function that casts the `void*` user data back to the object and calls a normal member function, keeping all logic inside the class.

System workflow

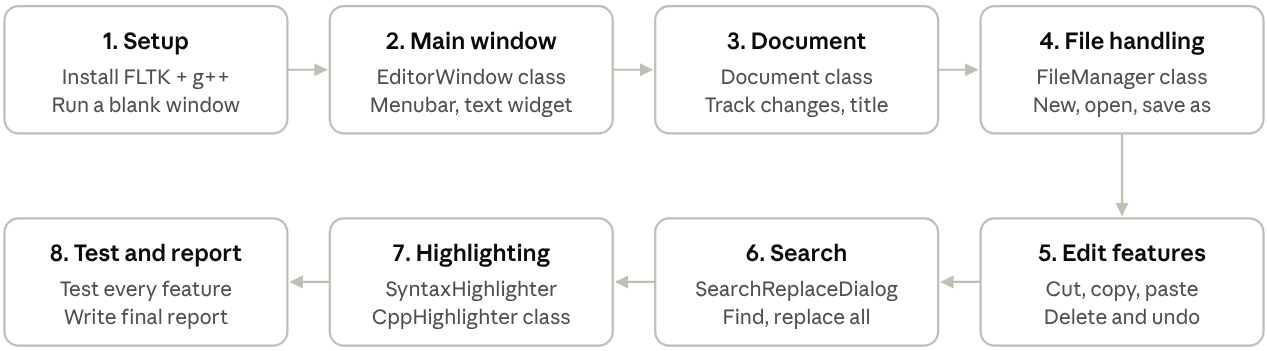
Every user action runs through the event loop and updates the document



On launch, `main()` creates the document and the editor window, then hands control to FLTK's event loop. Each menu choice or shortcut runs a callback that changes the document, and the title bar shows "(modified)" until the file is saved. Before exiting or starting a new file, the editor checks for unsaved changes and asks the user to save, discard or cancel.

Development workflow

Eight phases, each ending in a working build



The project is built in eight phases, one class or feature group at a time, so every phase ends with a program that compiles and runs. Each phase is committed to Git before the next begins, and the features added so far are tested before moving on. Phase 8 covers full testing of every menu item and edge case (empty search, unreadable file, cancel on save) and writing the final report.

Expected outcome and future enhancements

The final deliverable is a working cross-platform text editor that opens, edits, searches and saves text files, protects unsaved work, and highlights C/C++ code. The source code will be organised into separate header and source files per class, accompanied by a project report.

Possible future enhancements:

- Multiple documents in tabs
- Line numbers and a status bar showing cursor position
- Highlighters for more languages, such as Python or Java, added as new `SyntaxHighlighter` subclasses
- Case-sensitive and whole-word search options
- Recent files list and auto-save